

LAB REPORT

Lab Title	Pointer Notations of Arrays	Lab No.	10
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/4/28		
Lab description/objectives: 1. Demonstrate understanding of pointer notations of arrays in C programming. 2. Implement sorting functions in pointer notations as defined in the lab tasks/requirements. 3. Analyze critical aspects.			
Source code: BubbleSort: <pre>#include <stdio.h> #define MSize 10 typedef float Elementype; void bubbleSort(Elementype *arrPtr, int numel, int (*func)(Elementype, Elementype)); int dec(Elementype a, Elementype b); int inc(Elementype a, Elementype b); void swap(Elementype *a, Elementype *b); int main() { Elementype num[MSize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8}; bubbleSort(num, MSize, dec); for (int i = 0; i < MSize; i++) { printf("%.1f ", *(num + i)); } printf("\n"); bubbleSort(num, MSize, inc); for (int i = 0; i < MSize; i++) { printf("%.1f ", *(num + i)); } printf("\n"); return 0; } int dec(Elementype a, Elementype b) { return a < b; }</pre>			

```

int inc(Elementtype a, Elementtype b)
{
    return  $b < a$ ;
}

void bubbleSort(Elementtype *arrPtr, int numel, int (*func)(Elementtype, Elementtype))
{
    int end = numel;
    while (end--)
    {
        for (int i = 0; i < end; i++)
        {
            if (func(*(arrPtr + i), *(arrPtr + i + 1)))
            {
                swap((arrPtr + i), (arrPtr + i + 1));
            }
        }
    }
}

void swap(Elementtype *a, Elementtype *b)
{
    Elementtype temp = *a;
    *a = *b;
    *b = temp;
}

SelectionSort:
#include <stdio.h>
#define MSize 10
typedef float Elementtype;
void selectionSort(Elementtype *arrPtr, int numel, int (*func)(Elementtype, Elementtype));
int dec(Elementtype a, Elementtype b);
int inc(Elementtype a, Elementtype b);
void swap(Elementtype *a, Elementtype *b);
int main()
{
    Elementtype num[MSize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    selectionSort(num, MSize, dec);
    for (int i = 0; i < MSize; i++)
    {
        printf("%.1f ", *(num + i));
    }
    printf("\n");
    selectionSort(num, MSize, inc);
    for (int i = 0; i < MSize; i++)

```

```

{
    printf("%.1f", *(num + i));
}
printf("\n");
return 0;
}
int dec(Elementype a, Elementype b)
{
    return a < b;
}
int inc(Elementype a, Elementype b)
{
    return b < a;
}
void selectionSort(Elementype *arrPtr, int numel, int (*func)(Elementype, Elementype))
{
    for (int i = 0; i < numel; i++)
    {
        int extidx = i;
        for (int j = i + 1; j < numel; j++)
        {
            if (func(*(arrPtr + extidx), *(arrPtr + j)))
            {
                extidx = j;
            }
        }
        swap(arrPtr + i, arrPtr + extidx);
    }
}
void swap(Elementype *a, Elementype *b)
{
    Elementype temp = *a;
    *a = *b;
    *b = temp;
}

```

Program output:

```

10.0 9.9 8.8 7.7 6.6 5.5 4.4 3.3 2.2 1.1
1.1 2.2 3.3 4.4 5.5 6.6 7.7 8.8 9.9 10.0

```

Analysis:

1. Key Challenges & Solutions

- Understand the pointers to the function as the parameter of another function to use different function in a function.

- Understand method of the offset of the pointer to represent the elements in the array.
- Understand the logic of decreasing and increasing.

2. Result Validation

- Print the element in the array after sorting to ensure the correctness of the result.
- Note the format of output to make the result more readable.

3. Improvement Suggestions

- Use the *Elementtype* to typedef *float* to make the code more sustainable.
- Separate the *swap()* function to make the code more clear.

Conclusion:

When using the pointer notation to represent array elements, we need to understand the offset and the order of taking value and moving the iteration. When using the pointer to the function, we need to understand the logic and the storage of the function in the memory and its usage in other functions.